```
Welcome, {{name}}
```

HTML Entity Encoding

Blade and Laravel helper double-encode HTML entities by default. Call the **`Blade::withoutDoubleEncoding`** function from your **`AppServiceProvider's boot`** method to stop double encoding:

```
<?php
namespace App\Providers;
use Illuminate\Support\Facades\Blade;
use Illuminate\Support\ServiceProvider;
class AppServiceProvider extends ServiceProvider
{
 /*
  * Bootstrap any application services.
  */
 public function boot(): void
 {
  Blade::withoutDoubleEncoding();
 }
}
```

Components

Sections, layouts, and inclusions all give comparable benefits; nevertheless, some people may find the mental model of components and slots simpler to grasp. Class-based components and anonymous components are the two methods of writing components.

You may use the **`make:component`** Artisan command to construct a class-based component. We'll make an **`Alert`** component to demonstrate how to use components. The component will be placed in the **`app/View/Components`** directory using the **`make:component`** command:

```
php artisan make:component Alert
```

A view template will also be produced for the component using the **`make:component`** command. The **`resources/views/components`** directory will house the view. No further component registration is normally needed when creating components for your own application because they are